

Prime ATP on the pc-xp Frontier

Chaining, resolution replay, and a real-world search audit

MeTTapedia

July 2026

Abstract

We test whether MeTTa-Prime can both find proofs itself and independently check proofs found by other theorem provers. The test uses the implication/negation frontier that motivated Nil Geisweiller’s public **pc-xp** chaining experiments. The executable corpus is regenerated directly from a pinned, CC0 **set.mm** (the Metamath Proof Explorer’s database of machine-checked mathematics): 182 provability assertions from **ax-mp** through **ja**, with active essential hypotheses and source order preserved. On the full prior theory, Prime’s guided inhabitation search independently proves frontier theorems 181 and 183, but does not prove theorem 182 at the tested bounds. A chronological selector, trained only on the source prefix preceding each target, reduces the three frontier theories to 11, 11, and 12 assertions; Prime then discovers and rechecks all three proofs in 0.10 seconds. E and Vampire prove all twelve corresponding first-order encodings. Prime also replays their actual derivations: an E proof of theorem 182 becomes twelve independently checked binary-resolution steps and a direct proof term composed from eight earlier rules, which Prime’s ordinary type checker accepts. E rewriting and Vampire ordered superposition also replay against an independently checked equality certificate. A quantified extension checks the authentic **set.mm** theorem **ax5d**, which states $\varphi \rightarrow (\psi \rightarrow \forall x \psi)$ under the distinct-variable (DV) condition that x stays apart from ψ : with DV obligations hidden from the producer, Prime rediscovers the constructive core (**a1i ax-5**) within a two-assertion selected theory, then a source-level replayer infers the substitutions and recomputes the required **\$d x ps** condition. Nine destructive source-certificate mutations and one searched-term mutation are rejected.

The experiment also exposed two correctness bugs: a cyclic substitution accepted by an internal search unifier, and an unsound tautology test that confused unifiability with literal identity. Both fixes have positive and falsifying regression tests, and the complete Prime test suite passes. Prime remains substantially behind mature automated theorem provers in full-theory search. The results separate established capabilities from remaining work: proof checking and interchange succeed; small native resolution works; direct search has useful classical pruning; full-theory saturation still needs the standard redundancy and scheduling machinery that makes an ATP an ATP.

1 What we built and why it matters

We turned one concrete Metamath frontier into a controlled comparison of direct proof search, established first-order automated theorem provers (ATPs), and independent proof replay. For each target theorem, the available library is every earlier provability assertion in **set.mm** together with the target’s active hypotheses. Prime attempts to construct a typed proof term from that full library. E and Vampire receive first-order encodings generated from the same statements. Their successful derivations are then translated into explicit resolution or ordered-equality steps and replayed by Prime.

This setup answers four different questions:

1. Can Prime discover direct proofs when the useful lemmas are mixed into a realistically growing theory?
2. When its native search fails, can Prime validate a proof found by a mature ATP without trusting that prover’s success status?
3. Does the failure come from the theorem, the available rules, the proof calculus, or the search policy?
4. Can a selector discover a small useful theory from prior proof history without consulting the target’s stored proof?

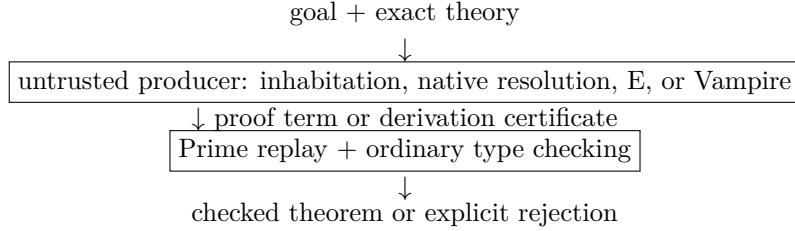
The headline result is sharp. Prime’s full-theory search proves theorems 181 and 183, while theorem 182 remains incomplete at the tested bounds. Chronological premise selection closes that gap: Prime searches and rechecks all three frontier proofs from small theories chosen solely from each earlier source prefix. E and Vampire solve all twelve generated comparison problems—four encodings for each of the three targets. For theorem 182, Prime also replays E’s twelve-step resolution derivation and checks the resulting direct proof term built from eight earlier rules. Thus

premise selection, proof interchange, and checking all work across the frontier; unguided full-library scaling remains the bottleneck.

The experiment grows out of Geisweiller’s public `pc-xp` work on rediscovering Metamath proofs and controlling backward search as the available theory expands [4, 5, 6]. Bitseat’s attention-allocation experiments provide a complementary selected-theory setting [7]. The benchmark itself is generated from the CC0 `set.mm` source [3], the single database from which the Metamath Proof Explorer machine-checks mathematics from propositional logic through ZFC set theory; the present frontier lies in its opening implication/negation segment.

Online implementation and artifact. The public [CeTTa Prime branch](#) contains the runtime, checker, search policies, and imported ATP libraries used here [1]. The corresponding [Mettapedia experiment directory](#) contains the source-derived problems, generators, replay fixtures, mutation tests, and executable gates; the [paper PDF](#) and [TeX source](#) are published on the same branch [2].

The implementation keeps three jobs separate:



Search may be heuristic, learned, incomplete, or simply wrong. It does not acquire theorem authority by being fast. Conversely, the checker need not implement the search strategy. This boundary lets us compare calculi and policies without weakening acceptance to accommodate any of them.

Section 2 fixes the benchmark. Section 3 reports direct search and chronological premise selection. Section 4 records two correctness bugs the corpus itself exposed. Sections 5–8 cover the external ATP differential, replay of E and Vampire derivations, ordered equality, and the native MeTTa resolution prototype. Section 9 draws the architecture boundary, and Section 10 extends the pipeline to a quantified theorem with authentic distinct-variable checking.

2 The `set.mm` frontier benchmark

The generator reads a pinned `set.mm` at revision `47e6e06b`. Every generated problem and its manifest carry the exact source identity:

```

revision = 47e6e06b87581cd630d210dc41cf83b02eea78ea
sha256   = 3aecffcfcab6f6e114cce1d873a8300d7f41f24928648c04164aaa305b1f491a

```

It parses the implication/negation grammar, tracks Metamath scopes and active essential hypotheses, and extracts the exact provability sequence from `ax-mp` to `ja`. Two syntax constructors precede the historical proof labels, so the resulting 182 assertions occupy indices 2–183. Anchor checks fail generation unless indices 2, 3, 4, 5, 6, 181, 182, and 183 are respectively `ax-mp`, `ax-1`, `ax-2`, `ax-3`, `mp2`, `pm2.61nii`, `pm2.61iii`, and `ja`.

For a theorem at index i , the *full prior theory* contains exactly every provability assertion at an index below i , plus the theorem’s grounded essential hypotheses. The generator also emits two controls:

- *core*: only `ax-mp`, `ax-1`, `ax-2`, and `ax-3`—modus ponens plus the three axiom schemes $\varphi \rightarrow (\psi \rightarrow \varphi)$, $(\varphi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \chi))$, and $(\neg\varphi \rightarrow \neg\psi) \rightarrow (\psi \rightarrow \varphi)$;
- *reference*: only labels occurring in the stored Metamath proof. This provides an oracle ceiling for premise selection.

The same extracted assertion has a direct typed-rule form and a first-order TPTP form.¹ For example, `pm2.61iii` is represented schematically as:

```

; Direct Curry--Howard rule
(: pm2.61iii
  (-> (impl (neg $ph) (impl (neg $ps) $ch))
      (impl $ph $ch)

```

¹TPTP is the standard problem language consumed by systems such as E and Vampire. TSTP extends that syntax with named inference steps and parent links, representing the returned derivation as a replayable DAG [8].

```

      (impl $ps $ch)
      $ch))
(chaining-rule pm2.61ii)

% First-order derivability encoding
fof(rule_pm2_61ii, axiom,
  ! [PH,PS,CH] :
    ((proved(imp(neg(PH),imp(neg(PS),CH)))
      & proved(imp(PH,CH))
      & proved(imp(PS,CH))) => proved(CH))).

```

In ordinary notation, `pm2.61ii` is the rule: from $\vdash \neg\varphi \rightarrow (\neg\psi \rightarrow \chi)$, $\vdash \varphi \rightarrow \chi$, and $\vdash \psi \rightarrow \chi$, infer $\vdash \chi$. The first form asks for a proof term inhabiting the target type. The second turns derivability into a first-order predicate so that a saturation prover can refute the axioms plus the negated target. They express the same rule application but expose very different search spaces.

3 Direct chaining in Prime

3.1 Readable syntax and exact checking

Prime’s direct producer consumes ordinary typing declarations. A rule joins search only when explicitly marked; open declarations do not silently become proof rules.

```

(: ph Type)  (: ps Type)  (: ch Type)
(: h1 ph)
(: h2 ps)
(: h3 (impl ph (impl ps ch)))

(: ax-mp (-> $a (impl $a $b) $b))
(chaining-rule ax-mp)

!(search-first-inhabitant
  &self ch 3 100000
  (search-policy atp-guided-inhabitation))
; checked answer: (ax-mp h2 (ax-mp h1 h3)) : ch

```

In ordinary notation: from φ , ψ , and $\varphi \rightarrow (\psi \rightarrow \chi)$, two modus-ponens steps derive χ . The producer instantiates a rule conclusion against the goal, creates its premises as subgoals, and builds the application term. Before returning a candidate, the ordinary Prime typing path checks the term against the target. The imported `lib_atp prototype` expresses the same direct construction using the runtime’s ordinary occurs-checked `unify`, together with `match` and `check-type`.

The current performance lane is an explicit search policy, not checker semantics. Its data-free controls include head filtering, term and formula-shape weights, premise cost, exact rigid-fact support, a 5:1 weight/age schedule, true fail-first premise ordering, and alpha-variant ancestor regularity (no open goal may repeat an ancestor goal up to variable renaming). Every control changes discovery order only.

3.2 Full-prior results at 181–183

The three frontier targets, in ordinary notation, are:

- `pm2.61nii` (181): from $\varphi \rightarrow (\psi \rightarrow \chi)$, $\neg\varphi \rightarrow \chi$, and $\neg\psi \rightarrow \chi$, infer χ ;
- `pm2.61iii` (182): from $\neg\varphi \rightarrow (\neg\psi \rightarrow (\neg\chi \rightarrow \theta))$, $\varphi \rightarrow \theta$, $\psi \rightarrow \theta$, and $\chi \rightarrow \theta$, infer θ ;
- `ja` (183): from $\neg\varphi \rightarrow \chi$ and $\psi \rightarrow \chi$, infer $(\varphi \rightarrow \psi) \rightarrow \chi$.

Table 1 reports the full-prior runs. The times and resident memory are observations from one development machine, not portable performance claims. All three queries use depth 12. Theorem 181 uses a 250,000 fuel bound; theorem 183 uses a 10,000,000 bound but finds its proof after a small fraction of it. Theorem 182 is shown at 250,000 fuel so its explicit incomplete receipt is directly comparable and inexpensive to reproduce.

The two independently discovered proof terms are:

```

pm2.61nii:
  (pm2.61ii pm2.24
    (pm2.61d1 h1 h3) h2)

ja:

```

Index	Theorem	Full-prior result	Work items	Time	Peak RSS
181	<code>pm2.61nii</code>	checked full-prior proof	607	0.60s	12.6 MiB
182	<code>pm2.61iii</code>	bounded incomplete (fuel exhausted)	6,388	6.10s	26.4 MiB
183	<code>ja</code>	checked full-prior proof	8,449	4.90s	30.7 MiB

Table 1: Prime guided inhabitation over the complete prior theory.

```
(pm2.61d2
 (imim12i notnotr h2) h1)
```

These runs search all 179 or 181 prior assertions. A larger 10,000,000-fuel stress run for theorem 182 still produced no proof within 180 seconds. Coverage is therefore nonmonotone at this frontier: Prime solves 181 and 183 while retaining a reproducible hole at 182.

The selected-support configuration solves 181–183 quickly. It provides an oracle ceiling and demonstrates that theory selection transfers between engines; the full-prior configuration measures premise discovery.

3.3 Chronological selection closes the three-theorem frontier

The oracle control is not the final selection result. We also built a deterministic selector whose training data for theorem i end at theorem $i - 1$. It starts with six recent assertions and the five assertions used most often by earlier source proofs. Because connective symbols alone cannot distinguish this implication/negation corpus, it then performs one bounded goal-directed relevance expansion. Candidate conclusions are unified with an open goal, and premise support is counted only when the premises join the target hypotheses under one shared substitution. Of at most four root candidates, only those tying for the fewest unsupported premises may add one earlier lemma whose instantiated premises are already target hypotheses. These operations select a theory; they do not accept a theorem.

The selector is tested for chronological isolation. Replacing the target’s stored proof, or replacing every proof at and after the target, must leave the selected labels unchanged. A cyclic selector substitution is also rejected. Prime then performs its normal guided inhabitation search over the selected theory, and a separate `check-type` call replays the returned term. The [selector source](#), [generated selection receipt](#), and [end-to-end gate](#) are directly browsable.

Index	Theorem	Selected assertions	Replayed proof term
181	<code>pm2.61nii</code>	11	<code>pm2.61i (pm2.61d1 h1 h3) h2</code>
182	<code>pm2.61iii</code>	11	<code>pm2.61i h4 (pm2.61ii h1 (a1d h2) (a1d h3))</code>
183	<code>ja</code>	12	<code>pm2.61d1 (imim2i h2) h1</code>

Table 2: Proofs discovered after strictly chronological premise selection.

All three searches and replays complete together in 0.10 seconds at 11.7 MiB peak resident memory on the same development machine. The relevance expansion is load-bearing for `ja`: the chronological seed contains `pm2.61d1`, and its instantiated open premise selects the much older `imim2i`. For theorem 182, the selector recovers a sufficient eleven-assertion theory from prior history and the target statement; Prime constructs the proof term shown in Table 2. This is premise discovery followed by proof discovery, whereas the reference-support control is only an oracle ceiling.

4 Correctness bugs found by theorem proving

Correctness falsifiers preceded performance attribution. Two of them found defects that ordinary happy-path tests had missed.

4.1 Finite substitutions require an occurs check

The first bug allowed an internal shape unifier to solve a type variable as a term containing itself. In the concrete regression, the rules below could fabricate an inhabitant of an unrelated `Target` through a cyclic equation equivalent to $X = \neg X$:

```
(: pm2.24ii (-> $ph (neg $ph) $ps))
(: pm2.18i  (-> (impl (neg $ph) $ph) $ph))
(: notnotr  (impl (neg (neg $ph)) $ph))
(: notnot   (impl $ph (neg (neg $ph))))
```

In ordinary notation these rules are: from φ and $\neg\varphi$ infer ψ ; from $\vdash \neg\varphi \rightarrow \varphi$ infer $\vdash \varphi$; and the axioms $\neg\neg\varphi \rightarrow \varphi$ and $\varphi \rightarrow \neg\neg\varphi$. The corrected path performs finite occurs-checked unification. The suite now requires all three outcomes:

```
AcyclicSearchAccepted
CyclicSearchRejected
CyclicReplayRejected
```

It is not enough for the producer to reject the cycle: direct replay of the pseudo-proof must also fail at the authority boundary.

4.2 Unifiability is not tautology

The native resolution prototype initially deleted any clause containing opposite-polarity atoms that could unify. That is unsound. The clause $P(X) \vee \neg P(Y)$ is not a tautology when X and Y are independently quantified; $P(X) \vee \neg P(X)$ is. The fixed deletion test requires exact alpha-aware complement identity, not the existence of a unifier:

```
(tautology ((ATP.Pos (p $x)) (ATP.Neg (p $x)))) ; True
(tautology ((ATP.Pos (p $x)) (ATP.Neg (p $y)))) ; False
(tautology ((ATP.Pos (p $x)) (ATP.Neg (p (f $x))))) ; False
```

This is why a large mathematical corpus is a correctness instrument as well as a benchmark: a small local error eventually manufactures many false theorems. Both bugs now have positive controls, falsifying negatives, and full-gate coverage.

5 External ATP differential

The generated TPTP suite contains four forms for each frontier theorem: semantic propositional, Hilbert core, Hilbert reference-support, and Hilbert full-prior-theory. E and Vampire both report **Theorem**—the success value in the SZS result vocabulary that ATPs emit—for all twelve problems (Table 3). The reference-support column is again an oracle control; the full-prior column is the meaningful search comparison.

	Semantic	Hilbert core	Hilbert reference	Hilbert full
pm2.61nii (181), E/Vampire	T/T	T/T	T/T	T/T
pm2.61iii (182), E/Vampire	T/T	T/T	T/T	T/T
ja (183), E/Vampire	T/T	T/T	T/T	T/T

Table 3: SZS outcomes for independently generated TPTP problems.

The differential changes the diagnosis of theorem 182. It is not intrinsically hard at this scale and it is not blocked by the available rule set. Its difficulty for Prime depends on problem form, calculus, indexing, redundancy control, and scheduling. E and Vampire provide working targets against which each missing component can be tested separately [9, 10].

6 From E’s TSTP refutation back to a Prime proof

An SZS status is useful evidence but not a certificate. The replay pipeline therefore consumes E’s actual TSTP proof for the full-prior theorem-182 problem. Its [TSTP-to-Prime generator](#) and [live E/Vampire replay gate](#) are part of the public artifact. The standard-library-only generator:

1. follows the empty clause’s parent DAG;
2. treats E’s classified descendants as the resolution leaves;

3. standardizes variables, independently recomputes every binary unifier with an occurs check, and verifies the exact resolvent;
4. expands E's nested **spm** (paramodulation) and **sr** (simplify-reflect) records into explicit binary-resolution certificates;
5. carries proof holes through the refutation and extracts a direct Hilbert proof term; and
6. emits deterministic MeTTa replay data.

The Python reader is only a generator. It is not trusted logic. Prime checks each generated resolution claim, and the extracted term must separately pass ordinary type checking. A representative replay step has an explicit finite substitution, selected pivots, two parent clauses, and the exact child:

```
!(assertEqual
  (atp:resolution:check-step
    &left-parent &right-parent
    (ATP.Subst ((ATP.Bind x term-x) (ATP.Bind y term-y)))
    (ATP.Neg (proved (imp (ATP.Var x) (ATP.Var y))))
    (ATP.Pos (proved (imp term-x term-y)))
    &expected-child)
  True)
```

For **pm2.61iii**, the live E proof reproduces a frozen twelve-step binary-resolution spine. Prime accepts all twelve steps, rejects a forged final resolvent, and checks the extracted term:

```
(pm2.61ii
  (syl6com (con1d hypothesis_1) hypothesis_3)
  (pm2.61i pm2.27
    (aid (con1i (con3i hypothesis_4))))
  hypothesis_2)
: f_th_182
```

The proof uses eight prior rules—**pm2.61ii**, **syl6com**, **con1d**, **pm2.61i**, **pm2.27**, **aid**, **con1i**, and **con3i**—plus the four theorem hypotheses. It is different from both the stored Metamath route and the small published support set that motivated the frontier. This is the desired ATP result: a new route found in the full theory, with every accepted conclusion grounded in a direct Prime proof.

The same pipeline replays E's four-step and Vampire's independently emitted resolution proofs for **ja**. Generator mutations alter pivots, parents, or children; replay mutations alter substitutions or resolvents. All are required to fail. Regeneration must be byte-for-byte deterministic, and live E/Vampire output must reproduce the frozen fixtures.

Scope of the conformance claim. The current replay begins at the CNF descendants used by the refutation; it does not claim a verified FOF-to-CNF transformation. This does not weaken the final theorem boundary because the reconstructed direct proof is checked against the original Prime declarations. A complete TSTP frontend should eventually certificate-check clausification as its own transformation.

7 Equality: ordered superposition as a second calculus

Resolution is sufficient for the present **pc-xp** fragment because it has no object-level equality. Equality needs ordered rewriting or superposition. The first equality rung uses the TPTP problem

```
fof(eq_ab, axiom, a = b).
fof(p_a, axiom, p(a)).
fof(goal, conjecture, p(b)).
```

In ordinary notation: from $a = b$ and $P(a)$, conclude $P(b)$. E reports the key inference as rewriting; Vampire exposes ordered superposition. Prime replays both as the same unit ordered-superposition certificate: selected equality, direction, target literal, term position, finite substitution, KBO (Knuth–Bendix ordering) configuration, and exact child. The configured KBO orients $b \succ a$, so the certified rewrite is $b \rightarrow a$ inside the negated goal. Prime then resolves with $P(a)$ and checks the constructive equality transport term.

Negative tests reject a reversed direction, changed child, changed position, reversed precedence, cyclic substitution, and rewriting at an original variable position. This is real ordered equality replay, but only for the unit-clause fragment. It is not yet a full superposition saturation prover; non-unit maximality, selection, factoring, simplification, and redundancy remain future work.

8 Native resolution in MeTTa: what works and what does not

The imported ATP library now contains a transparent binary-resolution producer written in MeTTa. The [resolution search library](#) and its [positive and falsifying examples](#) are directly inspectable. Its given-clause loop (the standard saturation main loop) already has:

- ordinary occurs-checked MeTTa unification and standardization apart;
- explicit set of support (inference stays anchored to goal-descended clauses);
- bounded local generation and bounded global steps;
- a 5:1 clause-weight/age schedule;
- exact tautology and alpha-variant deletion;
- a signed-literal Atomspace index; and
- a replayable resolution proof tree or a bounded-incomplete receipt.

For example, the universally schematic clauses $P(X) \vee Q(X)$, $\neg P(a)$, and $\neg Q(a)$ produce and replay the empty clause. The library also rejects a forged child and the cyclic unification of $P(X)$ with $P(f(X))$. Thus the inference rule, first-order variable handling, proof production, and bounded-result algebra are operational.

The real-world full-prior result is negative. Even the indexed native resolution run for theorem 181 was terminated without a result after 119 seconds at 22.4 GiB peak resident memory. The failure identifies the missing standard stack precisely:

1. forward and backward subsumption rather than exact-variant deletion;
2. demodulation and ordered inference restrictions;
3. compact persistent clauses and proof DAGs rather than large interpreted proof trees on every queued item;
4. discrimination/substitution-tree-quality indexing; and
5. a resumable, shared-waiter agenda instead of retaining duplicated recursive continuations.

These are classical base-prover mechanisms, not learned guidance. A PLN, neural ranker, or LLM should be measured only after this base is competent; otherwise the experiment mostly measures avoidable redundancy.

9 Architecture consequences for Prime

The experiment suggests a small and durable boundary.

Prime authority

Type elaboration, exact proof checking, subject/package identity, and replay of explicit certificate steps.

General MeTTa mechanism

Ordinary match/unify/substitution, indexed Atomspaces, and eventually a generic resumable stream/agenda handler with explicit bounds and ordering.

Imported proof libraries

Direct inhabitation, SLD/condensed detachment, resolution, superposition, premise selection, and PLN/learned ranking.

Replay and discovery share the same rule-application semantics. Replay pins the rule, substitution, and parents; search leaves those choices open and must schedule them. This is the useful sense in which checking is the fully-guided endpoint of proof search. A MeTTa-native controller can therefore rank ordinary proof-search choices and, later, PLN inference choices through the same library interface, while neither ranking policy acquires authority to accept its own result.

The present C `atp-guided-inhabitation` lane is useful as an explicit performance and correctness probe, but its scoring coefficients are not language semantics. The pure-MeTTa `lib_atp` implementation demonstrates the intended direction: proof search is readable, replaceable policy over ordinary language operations; the checker remains unchanged. If hot library operations later need compilation, that is an implementation refinement, not a reason to place one ATP calculus in the trusted core.

Direct inhabitation remains valuable: it constructs Curry–Howard proof terms and is natural for Horn-shaped theorem rules. Resolution remains valuable: it has mature redundancy theory, a fixed empty-clause target, and direct interoperability with E and Vampire. Prime should host both and compare them, not force one calculus to impersonate the other.

10 An authentic quantified `set.mm` theorem

The implication/negation frontier does not exercise Metamath’s distinct-variable (DV) side conditions. We therefore added a separate source replay for `ax5d`, which states

$$\vdash \varphi \rightarrow (\psi \rightarrow \forall x \psi) \quad \text{provided } x \text{ does not occur in } \psi,$$

where the proviso is exactly the source declaration `$d x ps`. Its stored proof is two steps: the `ax-5` instance $\vdash \psi \rightarrow \forall x \psi$, which carries the DV obligation, followed by `a1i`, the rule inferring $\vdash \varphi \rightarrow \theta$ from $\vdash \theta$. In Metamath source form (`$p` marks a provable assertion, `$=` its stored proof):

```
{
  $d x ps $.
  ax5d $p |- ( ph -> ( ps -> A. x ps ) ) $=
    ( wal wi ax-5 a1i ) BBCDEABCFG $.
}
```

The complete [quantified replay-and-search gate](#), [source elaborator](#), and [searched-term fixture](#) are available online.

The compact executable database is reconstructed from the pinned `set.mm`. Its compressed proof decodes to the reference sequence

```
wps wps vx wal wi wph wps vx ax-5 a1i
```

and hence to the two-node logical DAG `ax-5`; `a1i` described above. The grammar-driven (GSLT) source ledger—its Metamath grammar is supplied as data to a generic parsing engine, not hard-coded—preserves the floating hypotheses, scope, DV pair, theorem claim, and compressed payload. Its 25-object semantic database agrees exactly with an independent run of `mm-lean4`, a Lean 4-verified Metamath checker, before Prime sees the assertion data.

10.1 Search first, source semantics second

The constructive producer is deliberately ordinary typed inhabitation. It is given the two relevant assertions, but no proof term and no trusted DV answer:

```
(: Ax5dSearchVariable Type)
(: x Ax5dSearchVariable)
(: impl (-> Type Type Type))
(: all (-> Ax5dSearchVariable Type Type))
(: ph Type) (: ps Type)

(: ax-5 (impl $formula (all $variable $formula)))
(type-scheme ax-5)
(: a1i (-> $formula (impl $antecedent $formula)))
(chaining-rule a1i)

!(search-first-inhabitant &self
  (impl ph (impl ps (all x ps))) 3 1000000)
; found: (a1i ax-5)
```

This search erases DV obligations only from the producer. The exact returned term is compiled into source applications of `ax-5` and `a1i`:

```
(MMReplayApplyStep "ax5" "ax-5"
  (Cons (MMReplayBinding "ph" (ax5d:fff (ax5d:ps)))
    (Cons (MMReplayBinding "x" (ax5d:setvar "x")) Nil))
  Nil)

(MMReplayApplyStep "root" "a1i"
  (Cons (MMReplayBinding "ph" (ax5d:fff (ax5d:ax5-body)))
    (Cons (MMReplayBinding "ps" (ax5d:fff (ax5d:ph))) Nil))
  (Cons "ax5" Nil))
```

Replay does not trust those bindings. It checks their typecodes and source symbols, substitutes every mandatory hypothesis, compares the essential parent, substitutes the conclusion, and recomputes each source DV obligation. For `ax-5`, substituting $\varphi := \psi$ and $x := x$ turns the source condition “ x apart from φ ” (`$d x ph`) into the target obligation “ x apart from ψ ” (`$d x ps`). The theorem frame supplies exactly that pair.

This is Metamath’s flat token-sequence substitution discipline, not a claim that Prime now implements a general binder calculus. The source checker makes the quantified assertion sound by enforcing the same DV cross-product rule as Metamath: every variable occurring in one substituted expression must be distinct from, and declared disjoint from, every variable occurring in the other.

The positive proof and ten falsifiers are executable. Replay rejects a missing target DV pair, capture producing x/x , a wrong quantified variable, a changed parent, a changed substitution, a duplicate binding, and an undeclared symbol. It also rejects certificates that grant themselves an extra target-frame DV assumption or an invented target hypothesis. The producer bridge rejects a mutated search term instead of manufacturing a certificate for it. Thus the example demonstrates quantified proof discovery followed by authentic Metamath checking, while keeping search convenience outside the authority boundary.

11 What the real-world test establishes

1. **Prime can search, not only check.** Full-prior direct search independently proves 181 and 183 and returns exact proof terms.
2. **Chronological premise discovery works at the frontier.** A source-prefix-only selector reduces the three theories to 11, 11, and 12 assertions; Prime searches and rechecks all three, including theorem 182.
3. **Prime can consume mature ATP output.** Actual E and Vampire TSTP derivations replay through explicit resolution and ordered-equality checks.
4. **The hard frontier is now diagnostic.** Theorem 182 fails in the full-prior direct producer but succeeds after chronological selection and is also solved by E in the same full theory; Prime checks both resulting proof routes.
5. **Correctness testing paid for itself.** The corpus exposed an occurs-check bug and an unsound resolution deletion rule before either could contaminate performance claims.
6. **The boundary extends to a real quantified theorem.** Prime finds the DV-erased `ax5d` proof term in its two-assertion selected theory, then source replay independently reconstructs substitution and enforces the authentic `$d x ps` obligation.
7. **Native saturation is not competitive yet.** Small MeTTa resolution is correct and readable; the full theory demonstrates that subsumption, simplification, compact proof sharing, indexing, and scheduling are load-bearing.

The executable failures make the Prime ATP base useful for controlled comparison. Subsequent search policies can use the same CC0 problems, E/Vampire oracle outcomes, and final checker, holding theorem, theory, and acceptance constant.

12 Reproduction

Start from the [CeTTa Prime implementation branch](#) and the [Mettapedia reproducibility directory](#). The latter contains a [source-identity manifest](#), the generated TPTP/TSTP corpus, and every command named below. GitHub renders each link as a browsable source page; the gates regenerate and compare the executable data rather than relying on those rendered pages.

The main executable checks are:

```
# Prime correctness, cross-dialect, and mutation gates
make test-prime-all

# Regenerate Prime and TPTP problems from pinned set.mm
python3 setmm_prime_atp_tools.py \
  --setmm /path/to/set.mm \
  --source-revision 47e6e06b87581cd630d210dc41cf83b02eea78ea

# Live E + Vampire output, deterministic generators, Prime replay
CETTA=/path/to/cetta \
EPROVER=/path/to/eprover \
VAMPIRE=/path/to/vampire \
./run_setmm_prime_external_tstp_replay_v0_gate.sh

# Regenerate and check the 181 / 182 / 183 direct outcomes
CETTA=/path/to/cetta SETMM=/path/to/set.mm \
./run_setmm_prime_direct_frontier_v0_gate.sh

# Chronological selection, isolation mutations, search, and proof replay
CETTA=/path/to/cetta SETMM=/path/to/set.mm \
```

```

./run_setmm_prime_chronological_v0_gate.sh

# Quantified ax5d: source reconstruction, independent semantic agreement,
# DV-aware replay mutations, and DV-erased proof discovery
SETMM=/path/to/set.mm \
CETTA_PRIME=/path/to/cetta \
CETTA_GSLT=/path/to/gslt-enabled-cetta \
GSLT_GRAMMAR=/path/to/lib_parse_metamath_grammar_generated_v0.metta \
./run_setmm_prime_quantified_ax5d_v0_gate.sh

```

The external replay gate covers live E resolution for ja, live E resolution for pm2.61iii, independent Vampire resolution for ja, E rewriting, Vampire ordered superposition, deterministic regeneration, extractor mutations, replay mutations, and final Prime proof checks. Timing and stress logs are evidence artifacts, not correctness gates.

References

- [1] CeTTa contributors, *CeTTa Prime: checking-first ATP implementation and tests*, snapshot [ed24caa](#).
- [2] MeTTapeda contributors, *Prime ATP on the pc-xp frontier: executable artifact and paper*, public branch, <https://github.com/zariuq/MeTTapeda/tree/prime-atp-pcxp-20260718>.
- [3] The Metamath community, *Metamath Proof Explorer (set.mm)*, CC0 1.0, <https://github.com/metamath/set.mm/tree/47e6e06b87581cd630d210dc41cf83b02eea78ea>.
- [4] N. Geisweiller, *Chaining experiments for MeTTa*, <https://github.com/ngeiswei/chaining>.
- [5] N. Geisweiller, *Inference Control in MeTTa*, AITP 2024, https://aitp-conference.org/2024/abstract/AITP_2024_paper_2.pdf.
- [6] N. Geisweiller, *Using Forward Chaining to Go Backward*, 2026, <https://github.com/ngeiswei/papers/blob/master/2026/BackwardViaForward/BackwardViaForward.pdf>.
- [7] Bitseat, *Metamath attention-allocation experiments*, public branch, <https://github.com/Bitseat/chaining/tree/metamath-aa/experimental/metamath-aa>.
- [8] G. Sutcliffe, *TPTP Format for Derivations*, <https://tptp.org/UserDocs/QuickGuide/Derivations.html>.
- [9] S. Schulz et al., *The E Theorem Prover*, <https://www.e prover.org/>.
- [10] L. Kovács and A. Voronkov et al., *The Vampire Theorem Prover*, <https://vprover.github.io/>.
- [11] J. A. Robinson, “A Machine-Oriented Logic Based on the Resolution Principle,” *Journal of the ACM* 12(1), 1965, pp. 23–41.
- [12] L. Bachmair and H. Ganzinger, “Resolution Theorem Proving,” in *Handbook of Automated Reasoning*, 2001, pp. 19–99.